block(width: 100%, outset: (x: 25mm), fill: navy, inset: (top: 18mm, bottom: 18mm),)[

TP 01

Démarrage du noyau Linux

Initramfs, système de fichiers racine et QEMU

AUTEUR
Lionel Auroux

DATE
2026

DURÉE
5h

]

Sommaire

Objetifs 3
Construire un environnement Busybox 3
    Construire une initramfs from scratch 4
    Compiler Linux 6
    Démarrer QEMU 6
    Débogage avec QEMU 6
RootFS complet — Système Linux minimal sur QEMU 7
    1.1 : Image disque 7
    1.2 : Peuplement de l'image sous Docker 8
    1.3 : Installation de GRUB dans l'image 9
[Optionnel] RootFS via NFS 11

Objetifs

L'objectif de cet atelier est de faire tourner un Linux personnalisé de 3 manières différentes avec QEMU :

- un système busybox avec une initramfs (optionnel)
- un vrai LFS (*Linux From Scratch*) (obligatoire)
- un noyau avec un rootfs via NFS (optionnel)

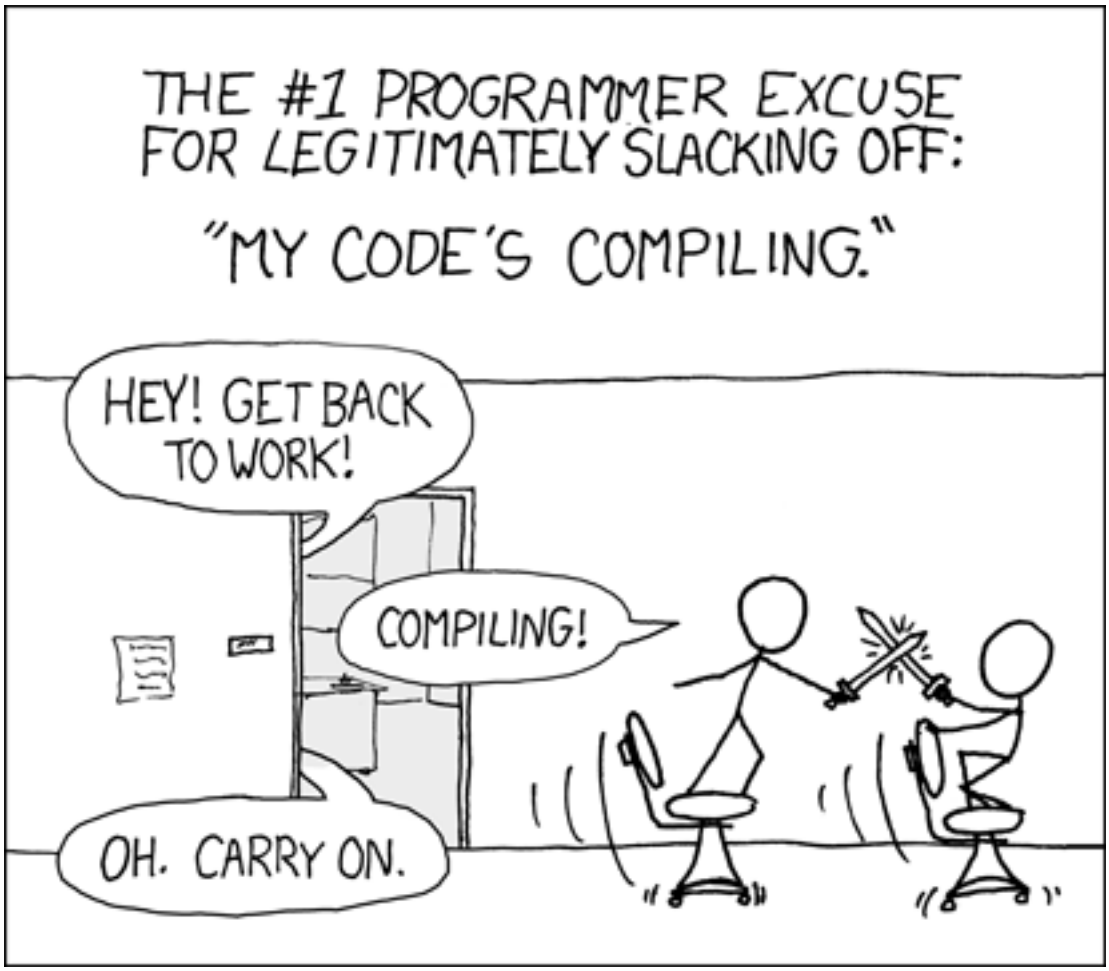


Fig. 1. – width:400px

Construire un environnement Busybox

Busybox est un espace utilisateur minimaliste. Il fournit des versions allégées des outils usuels (`cd`, `cat`, `insmod`, etc.)

Il est recommandé de l'utiliser plutôt qu'une distribution Linux complète car il est plus simple à configurer et réduit le temps de démarrage.

```shell
user $ git clone git://git.busybox.net/busybox
user $ cd busybox
user $ make defconfig # Toutes les fonctionnalités, sans debug
user $ make           # Compilation
user $ make install   # Copie les fichiers générés dans _install/
```

Construire une initramfs from scratch

L'initramfs est le premier système de fichiers, chargé par le noyau en mémoire sur `/`. Linux ne requiert qu'un programme `init`, notre busybox et quelques bibliothèques.

À la fin du démarrage, on lancera un programme qui boucle indéfiniment en spawant un shell.

Dans un répertoire de travail `$BUILDS`, écrire un programme `my_init_loop/main.c` :

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    printf("ECOLE 2600 MY INIT\n");
    while (2600)
    {
        pid_t pid = fork();
        int status = 0;
        if (pid)
        {
            waitpid(pid, &status, 0);
            printf("Respawn\n");
            pid = 0;
        }
        else
        {
            char *tab[] = {"/usr/bin/setsid", "cttyhack", "sh", NULL};
            execv("/usr/bin/setsid", tab);
        }
    }
}
```

Ce programme sera exécuté dans notre système ainsi :

```shell
# Configuration des variables
user $ export BUILDS=$PWD
user $ export INITRAMFS_BUILD=$PWD/initramfs
```

```
user $ export BUSYBOX_BUILD=$PWD/busybox

# Construction du répertoire contenant notre initramfs
user $ mkdir -p $INITRAMFS_BUILD
user $ cd $INITRAMFS_BUILD
user $ mkdir -p bin sbin etc proc sys dev usr/bin usr/sbin

# Retour dans notre répertoire et copie de busybox
user $ cd $BUILDS
user $ cp -a $BUSYBOX_BUILD/_install/* $BUILDS/initramfs

# Compilation du programme init_loop qui boucle et spawn un shell
user $ gcc $BUILDS/my_init_loop/main.c -o $INITRAMFS_BUILD/init_loop
```

shell

```
# Copie de la bibliothèque C depuis le système hôte
user $ mkdir -p $INITRAMFS_BUILD/lib/x86_64-linux-gnu/
user $ mkdir -p $INITRAMFS_BUILD/lib64
user $ cp /lib/x86_64-linux-gnu/libc.so.6      $INITRAMFS_BUILD/lib/x86_64-
linux-gnu/
user $ cp /lib/x86_64-linux-gnu/libm.so.6      $INITRAMFS_BUILD/lib/x86_64-
linux-gnu/
user $ cp /lib/x86_64-linux-gnu/libresolv.so.2 $INITRAMFS_BUILD/lib/x86_64-
linux-gnu/
user $ cp /lib64/ld-linux-x86-64.so.2          $INITRAMFS_BUILD/lib64
```

- Les bibliothèques doivent se trouver là où le système les cherche. Ici, on suit la convention Debian/Ubuntu/Mint.
- Sur une autre distribution, se fier à `ldd init_loop` et `ldd bin/busybox`.

shell

```
# Script init lancé par le noyau au démarrage
user $ cat > $INITRAMFS_BUILD/init <<'END'
#!/bin/sh

mount -t proc none /proc
mount -t sysfs none /sys
mount -t devtmpfs none /dev

cat <<!


Boot took $(cut -d' ' -f1 /proc/uptime) seconds

 ____________                 .__                 ________  ________________  ________
 \_   _____/ ____  ____ |  |   ____   \_____  \/  _____/\    _  \ \    _  \
  |    __)__/ ___\/  _ \|  | _/ __ \   /  ____/   __  \ /  /_\  \/  /_\  \  \
  |        \  \__(  <_> )  |_\  ___/  /       \  |__\  \\  \_/   \  \_/   \
 /_______  /\___  >____/|____/\___  > \_______ \_____  / \_____  /\_____  /
         \/     \/                \/          \/     \/        \/       \/


Welcome to "Ecole 2600 linux"
!
```

```
/init_loop
END

user $ chmod +x $INITRAMFS_BUILD/init

# Construction de l'image CPIO gzippée
user $ cd $BUILDS/initramfs
user $ find . -print0 | cpio --null -ov --format=newc | \
        gzip -9 > $BUILDS/initramfs.cpio.gz
```

Il s'agit d'une initramfs basique.

Pour plus d'informations : https://www.kernel.org/doc/html/latest/filesystems/ramfs-rootfs-initramfs.html

Compiler Linux

Compilons un noyau pour tester notre système minimal :

shell

```
user $ sudo apt install libelf-dev   # libelf-dev sur Debian/Ubuntu...
user $ cd linux-XXX
user $ make defconfig
user $ make
```

Démarrer QEMU

shell

```
user $ qemu-system-x86_64 -kernel /path/to/linux/arch/x86/boot/bzImage \
        -initrd $BUILDS/initramfs.cpio.gz \
        -append "console=ttyS0" -nographic
```

On observe le noyau démarrer avec cette initramfs.

Débogage avec QEMU

- Compiler Linux avec les informations de debug (chercher l'option adéquate dans la configuration).
- Lancer QEMU avec un serveur gdb : `-s`
- Lancer gdb avec le binaire noyau : `gdb vmlinux`
- Dans gdb, se connecter à QEMU : `target remote localhost:1234`
- Poser des points d'arrêt : `b my_symbol`

▶ Lancer la simulation dans gdb : `continue`

RootFS complet — Système Linux minimal sur QEMU

Notre objectif final est de créer un Linux minimaliste (Linux From Scratch). Dans cet exemple, on construit une variante de la distribution Alpine, mais la démarche s'adapte à d'autres distributions.

1.1 : Image disque

On prépare une image disque pour le système. On crée un fichier vide de 450 Mo :

```shell
$ truncate -s 450M disk.img
```

Ce fichier représente un disque dur entier. On y crée une table de partitions BIOS :

```shell
$ /sbin/parted -s ./disk.img mktable msdos
```

On ajoute une partition primaire de type ext4 occupant tout l'espace :

```shell
$ /sbin/parted -s ./disk.img mkpart primary ext4 1 "100%"
```

On la marque comme bootable :

```shell
$ /sbin/parted -s ./disk.img set 1 boot on
```

On crée un périphérique loopback pour accéder à l'image et ses partitions :

```shell
$ sudo losetup -Pf disk.img
```

On note quel périphérique est associé à `disk.img` :

```shell
```

```
$ losetup -l
```

On formate la première partition en ext4 :

```shell
$ sudo mkfs.ext4 /dev/loopXp1
```

On crée un répertoire de travail et on monte la partition :

```shell
$ mkdir -p /tmp/my-rootfs
$ sudo mount /dev/loopXp1 /tmp/my-rootfs
```

On demande à Docker d'installer un Alpine Linux minimal dans notre répertoire de travail :

```shell
$ docker run -it --rm -v /tmp/my-rootfs:/my-rootfs alpine
```

1.2 : Peuplement de l'image sous Docker

Les étapes suivantes s'effectuent **sous Docker** après la commande `docker run`.

On configure Alpine avec un système minimal, des utilitaires de base et un compilateur C :

```shell
$ apk add openrc
$ apk add util-linux
$ apk add build-base
```

On configure un terminal pour l'accès série via QEMU :

```shell
$ ln -s agetty /etc/init.d/agetty.ttyS0
$ echo ttyS0 > /etc/securetty
# Terminal port série comme login par défaut
$ rc-update add agetty.ttyS0 default
$ rc-update add root default
```

On définit un mot de passe pour root :

```shell
$ passwd root
```

On active les pseudo-systèmes de fichiers au démarrage :

```shell
$ rc-update add devfs boot
$ rc-update add procfs boot
$ rc-update add sysfs boot
```

On copie la configuration Docker dans la partition montée :

```shell
$ for d in bin etc lib root sbin usr; do tar c "/$d" | tar x -C /my-rootfs;
done
```

On crée les répertoires spéciaux :

```shell
$ for dir in dev proc run sys var; do mkdir /my-rootfs/${dir}; done
```

On quitte Docker :

```shell
$ exit
```

1.3 : Installation de GRUB dans l'image

On peut éditer le message d'accueil d'Alpine :

```shell
$ sudo vim /tmp/my-rootfs/etc/issue
```

Il faut disposer d'un noyau Linux compilé à installer dans la partition.

On crée le répertoire de boot pour GRUB et le noyau :

```shell
```

```
$ sudo mkdir -p /tmp/my-rootfs/boot/grub
```

On copie le noyau compilé (depuis le répertoire `linux/`) :

```shell
$ sudo cp linux/arch/x86/boot/bzImage /tmp/my-rootfs/boot/vmlinuz
```

On crée et édite le fichier `grub.cfg` :

```shell
$ sudo vim /tmp/my-rootfs/boot/grub/grub.cfg
```

Contenu du fichier `grub.cfg` :

```code
serial
terminal_input serial
terminal_output serial
set root=(hd0,1)
menuentry "Linux2600" {
    linux /boot/vmlinuz root=/dev/sda1 console=ttyS0
}
```

On installe GRUB via le loopback pour le boot BIOS :

```shell
$ sudo grub-install --directory=/usr/lib/grub/i386-pc \
    --boot-directory=/tmp/my-rootfs/boot /dev/loopX
```

On démonte la partition et on détache le loopback :

```shell
$ sudo umount /tmp/my-rootfs
$ sudo losetup -d /dev/loopX
```

On lance QEMU sur l'image disque :

```shell
$ qemu-system-x86_64 -hda disk.img -nographic
```

[Optionnel] RootFS via NFS

Utiliser une image disque comme rootfs pose plusieurs problèmes :

- Il faut s'assurer que le contenu de l'image est à jour (Linux peut garder des modifications en cache).
- La VM démarrée utilise l'image en lecture seule ; modifier un fichier impose un redémarrage.

Un export NFS comme rootfs évite ces deux problèmes.

Étapes :

- Compiler un noyau avec le support NFS et le boot NFS.
- Créer un rootfs (celui de busybox est un bon point de départ).
- Configurer un serveur NFS (voir la documentation de votre distribution).
- Éditer `/etc/exports` pour exporter `/chemin/vers/votre/rootfs`. Selon la configuration, ajouter l'option `insecure`.
- Activer les exports : `exportfs -av`
- Mettre à jour la ligne de commande noyau dans `qemu` :

```shell
root=/dev/nfs nfsroot=10.0.2.2:/path/to/rootfs rw
ip=10.0.2.15::10.0.2.1:255.255.255.0
```

Notes :

- `root=/dev/nfs` est un chemin spécial demandant l'utilisation de NFS pour le rootfs.
- Par défaut, QEMU fournit un réseau en mode utilisateur (pas de privilèges administrateur). Certains protocoles réseau ne fonctionnent pas (ex : `ICMP` — pas de `ping` depuis la VM).
- `10.0.2.2` est l'adresse IP par défaut de la machine hôte, visible uniquement depuis la VM.
- `rw` indique que le système de fichiers est accessible en écriture.
- `ip=...` définit la configuration réseau de l'interface émulée.

```shell
user $ qemu-system-x86_64 -kernel /path/to/linux/arch/x86/boot/bzImage \
    -append "init=/linuxrc root=/dev/nfs nfsroot=10.0.2.2:./devnfs,vers=3 \
        rw ip=10.0.2.15::10.0.2.1:255.255.255.0 console=ttyS0
raid=noautodetect" \
        -nographic
```